



Автономная некоммерческая образовательная организация высшего образования
«Университет Иннополис»

ОТЧЕТ ПО ПРАКТИКЕ

Тема работы

**Построение и анализ ансамблевых моделей
на основе решающих деревьев: регрессия**

Выполнили:

Камалов Булат

Умаханов Мансур

Руководитель:

Корнаева Е.П.

Университет Иннополис

2026

Содержание

Введение	5
1. Постановка задачи и описание датасета	6
2. Разведочный анализ и предобработка данных	8
3. Базовая модель и проблема переобучения	16
4. Ансамблевые модели: Bagging и Random Forest	20
5. Градиентный бустинг: XGBoost	25
6. Сравнительный анализ моделей	30
Заключение	38
Список использованных источников	40

Список рисунков

Рисунок 1 - Фрагмент исходного датасета

Рисунок 2 - Распределение целевой переменной (charges)

Рисунок 3 - Диаграмма размаха целевой переменной (charges)

Рисунок 4 - Стоимость страховки в группах по признаку курения (smoker)

Рисунок 5 - Корреляционные матрицы: а) с (charges); б) с $\log(\text{charges} + 1)$

Рисунок 6 - Код стратифицированного разделения и фрагмент датасета после предобработки

Рисунок 7 - Первые три уровня оптимизированного дерева решений

Рисунок 8 - Метрики дерева до и после оптимизации

Рисунок 9 - Лучшие гиперпараметры оптимизированного дерева решений

Рисунок 10 - Остатки оптимизированного дерева на тестовой выборке

Рисунок 11 - Лучшие гиперпараметры Bagging

Рисунок 12 - Лучшие гиперпараметры Random Forest

Рисунок 13 - Сравнение оценок OOB, KFold и test

Рисунок 14 - Важность признаков Random Forest

Рисунок 15 - Лучшие гиперпараметры XGBoost

Рисунок 16 - Обоснование выбора 648 деревьев для XGBoost

Рисунок 17 - Остатки XGBoost на тестовой выборке

Рисунок 18 - Важность признаков XGBoost

Рисунок 19 - Сравнение моделей по R^2 на train и test

Рисунок 20 - Сравнение MAE и RMSE на тестовой выборке

Рисунок 21 - Предсказанные и фактические значения для всех моделей

Рисунок 22 - Остатки моделей на тестовой выборке

Рисунок 23 - Сравнение важности признаков

Рисунок 24 - SHAP-анализ модели Random Forest

Рисунок 25 - Среднее время инференса моделей

Список таблиц

Таблица 1 - Описательные статистики целевой переменной (charges)

Таблица 2 - Размеры итоговых выборок

Таблица 3 - Метрики моделей на train, validation и test

Введение

Актуальность работы обусловлена ростом объема данных в страховании и необходимостью точнее оценивать индивидуальный риск клиента. Методы машинного обучения позволяют учитывать совместное влияние демографических и поведенческих факторов, благодаря чему расчет страховой стоимости становится более обоснованным. [1]

Цель работы - построить и сопоставить модели регрессии для прогнозирования стоимости медицинской страховки (charges), а также определить признаки, которые сильнее всего влияют на результат. В качестве факторов рассматриваются возраст клиента (age), индекс массы тела (bmi), количество детей (children), пол (sex), факт курения (smoker) и регион проживания (region).

Исследование проводилось последовательно. После разведочного анализа и предобработки данных было построено одиночное дерево решений, затем ансамбли Bagging и Random Forest, а на заключительном этапе - модель XGBoost. Разделение на train, validation и test позволило отделить подбор гиперпараметров от итоговой оценки качества и проверить модели на переобучение.

1. Постановка задачи и описание датасета

Решается задача регрессии: по характеристикам клиента требуется предсказать непрерывную величину стоимости медицинской страховки (charges). Исходный датасет включает 1338 наблюдений и 7 столбцов, из которых шесть являются входными признаками, а один - целевой переменной. [9]

К количественным признакам относятся возраст клиента (age), индекс массы тела (bmi) и количество детей (children). Пол клиента (sex), факт курения (smoker) и регион проживания (region) представлены категориальными признаками. Фрагмент исходных данных приведен на рисунке 1.

Фрагмент исходного датасета

age	sex	bmi	children	smoker	region	charges
19	female	27.9	0	yes	southwest	16884.92
18	male	33.77	1	no	southeast	1725.55
28	male	33.0	3	no	southeast	4449.46
33	male	22.705	0	no	northwest	21984.47
32	male	28.88	0	no	northwest	3866.86
31	female	25.74	0	no	southeast	3756.62
46	female	33.44	1	no	southeast	8240.59

Рисунок 1 - Фрагмент исходного датасета

В исходном наборе одновременно присутствуют числовые и строковые значения. До передачи данных моделям категориальные столбцы необходимо преобразовать в числовую форму, сохранив содержательный смысл категорий.

Проверка целостности показала отсутствие пропущенных значений, поэтому заполнение данных не потребовалось. Вместе с тем выражение `(df.duplicated().sum())` обнаружило 1 полностью повторяющуюся строку. Она была удалена, и в дальнейшей работе использовались 1337 уникальных наблюдений.

```
missing_by_column = df.isna().sum()
duplicate_count = df.duplicated().sum()

df = df.drop_duplicates().reset_index(drop=True)
```

2. Разведочный анализ и предобработка данных

2.1. Выбор метрик качества

До построения моделей были определены показатели, по которым будет оцениваться результат. Использовалось несколько метрик, поскольку одна величина не отражает все стороны ошибки. Коэффициент детерминации (R^2) показывает долю вариации целевой переменной, объясненную моделью. Средняя абсолютная ошибка (MAE) характеризует типичное отклонение прогноза в долларах и сравнительно слабо зависит от единичных крупных промахов. [1], [2]

Среднеквадратическая ошибка (MSE) сильнее штрафует крупные ошибки, что важно для дорогих страховых случаев. Корень из среднеквадратической ошибки (RMSE) возвращает этот показатель в исходные единицы измерения. Средняя абсолютная процентная ошибка (MAPE) дополняет анализ относительной величиной ошибки, однако для небольших значений стоимости (charges) ее следует трактовать осторожно: деление на малое фактическое значение может непропорционально увеличить процентную ошибку.

$$R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2 \quad (1)$$

$$MAE = (1 / n) \Sigma |y_i - \hat{y}_i| \quad (2)$$

$$MSE = (1 / n) \Sigma (y_i - \hat{y}_i)^2, \quad RMSE = \sqrt{MSE} \quad (3)$$

$$MAPE = (100\% / n) \Sigma |(y_i - \hat{y}_i) / y_i| \quad (4)$$

Все метрики рассчитывались после обратного преобразования прогнозов в исходную шкалу стоимости. Их сопоставление на train, validation и test использовалось одновременно для ранжирования моделей и оценки устойчивости результатов.

2.2. Анализ распределения целевой переменной

После удаления повторяющейся строки были рассчитаны описательные статистики стоимости страховки (charges), представленные в таблице 1. Затем форма распределения и высокие значения были исследованы графически средствами Matplotlib и Seaborn. [6], [7]

Таблица 1 - Описательные статистики целевой переменной (charges)

Статистика	Значение
Среднее	13 279.12
Медиана	9 386.16
Мода	Не определяется: все значения уникальны
Дисперсия	146 660 811.01
Стандартное отклонение	12 110.36
Коэффициент вариации	91.2%
Минимум	1 121.87
Максимум	63 770.43

Стандартное отклонение (12 110.36 доллара) сопоставимо со средним значением (13 279.12 доллара), а коэффициент вариации равен 91.2%. Это характеризует высокий относительный разброс, но само по себе не определяет направление асимметрии. О правосторонней асимметрии свидетельствуют среднее, превышающее медиану (9 386.16 доллара), положительный коэффициент асимметрии и длинный хвост дорогих случаев до 63 770.43 доллара. Точная мода не выделяется, поскольку после удаления дубликата каждое значение (charges) встречается один раз; наиболее заполненной остается нижняя область гистограммы.

Распределение представлено на рисунке 2: основная часть наблюдений сосредоточена в нижнем диапазоне, тогда как небольшая группа дорогих случаев формирует длинный правый хвост.

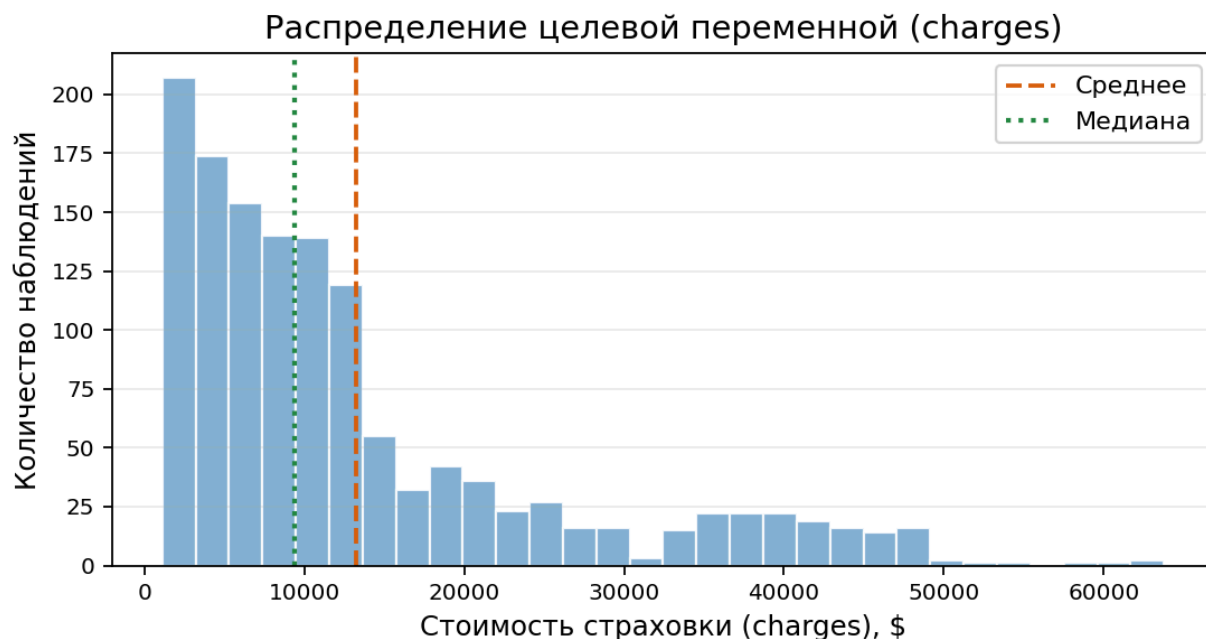


Рисунок 2 - Распределение целевой переменной (charges)

На гистограмме среднее (13 279.12 доллара) расположено правее медианы (9 386.16 доллара). Коэффициент асимметрии равен 1.52: положительное значение соответствует длинному правому хвосту, поэтому исходное распределение имеет выраженную правостороннюю асимметрию.

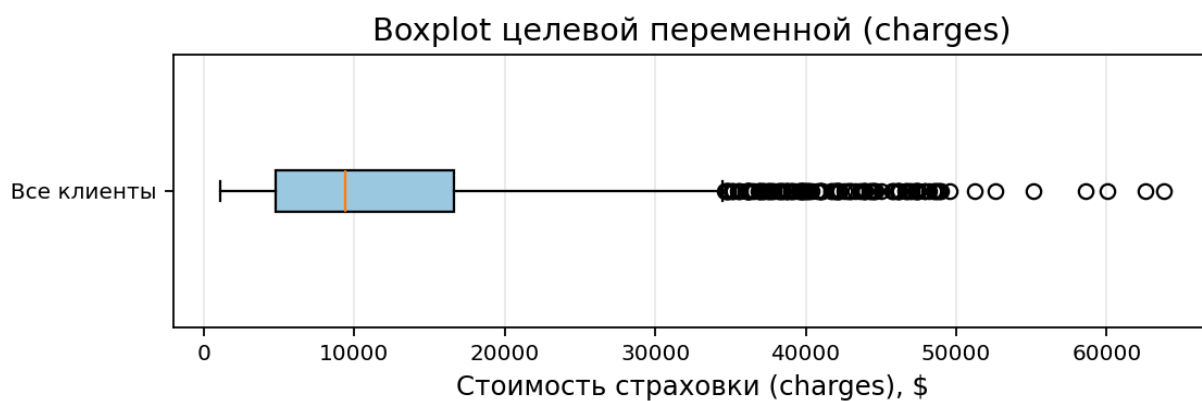


Рисунок 3 - Диаграмма размаха целевой переменной (charges)

Диаграмма размаха выделяет группу высоких значений за верхней границей. Однако положение точки за границей диаграммы само по себе не означает ошибку: необходимо проверить, объясняются ли такие значения характеристиками клиентов.

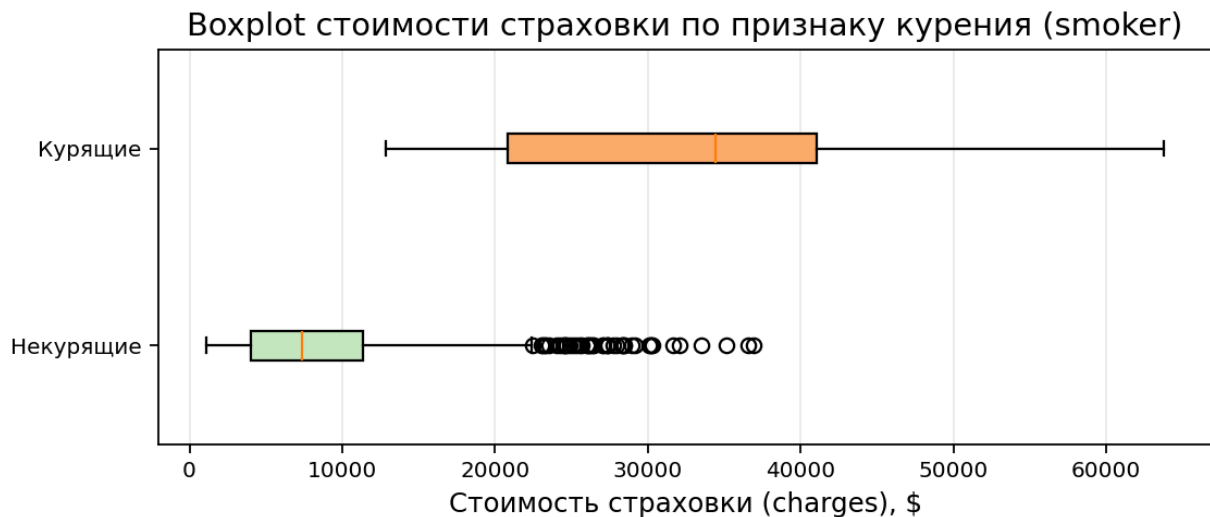


Рисунок 4 - Стоимость страховки в группах по признаку курения (smoker)

Стоимость выше 35 000 долларов имеют 133 объектов, или 9.9% выборки. Среди них 97.7% являются курильщиками. Поэтому большинство высоких значений отражает реальную закономерность для группы (smoker = yes), а не ошибки измерения; удалять их не следует.

Логарифмирование целевой переменной

Чтобы уменьшить влияние длинного правого хвоста, модель обучалась на логарифме стоимости. К исходной величине прибавлялась единица, что делает преобразование корректным и для нулевых значений. После прогноза выполнялось обратное преобразование, поэтому метрики рассчитывались в долларах. [5]

$$y_{\log} = \ln(y + 1) \quad (5)$$

$$\hat{y} = \exp(\hat{y}_{\log}) - 1 \quad (6)$$

```
df['charges_original'] = df['charges']
df['charges'] = np.log1p(df['charges'])

# обратное преобразование после прогноза
y_pred = np.expml(model.predict(X_test))
```

После логарифмирования коэффициент асимметрии снизился с 1.52 до -0.09. Распределение стало значительно ближе к симметричному, а редкие дорогие полисы перестали чрезмерно доминировать в функции потерь.

2.3. Анализ линейной зависимости признаков

Корреляции с исходной целевой переменной (charges)

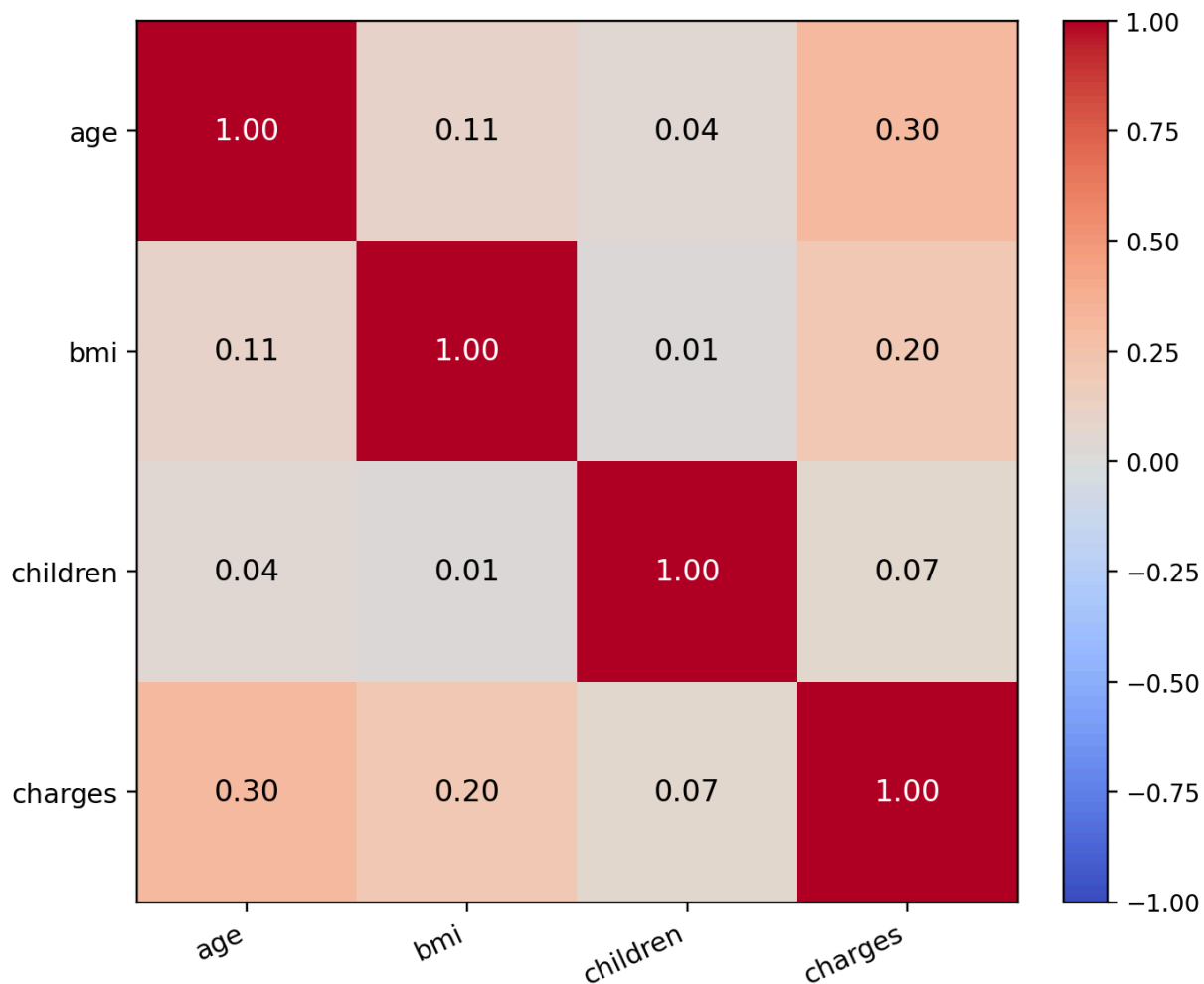


Рисунок 5, а - Корреляционная матрица с исходной целевой переменной (charges)

Для исходной стоимости наиболее заметна связь с возрастом клиента (age), однако ее коэффициент составляет только около 0,30. Между самими количественными признаками сильных линейных зависимостей не обнаружено.

Корреляции с логарифмом целевой переменной $\log(\text{charges} + 1)$

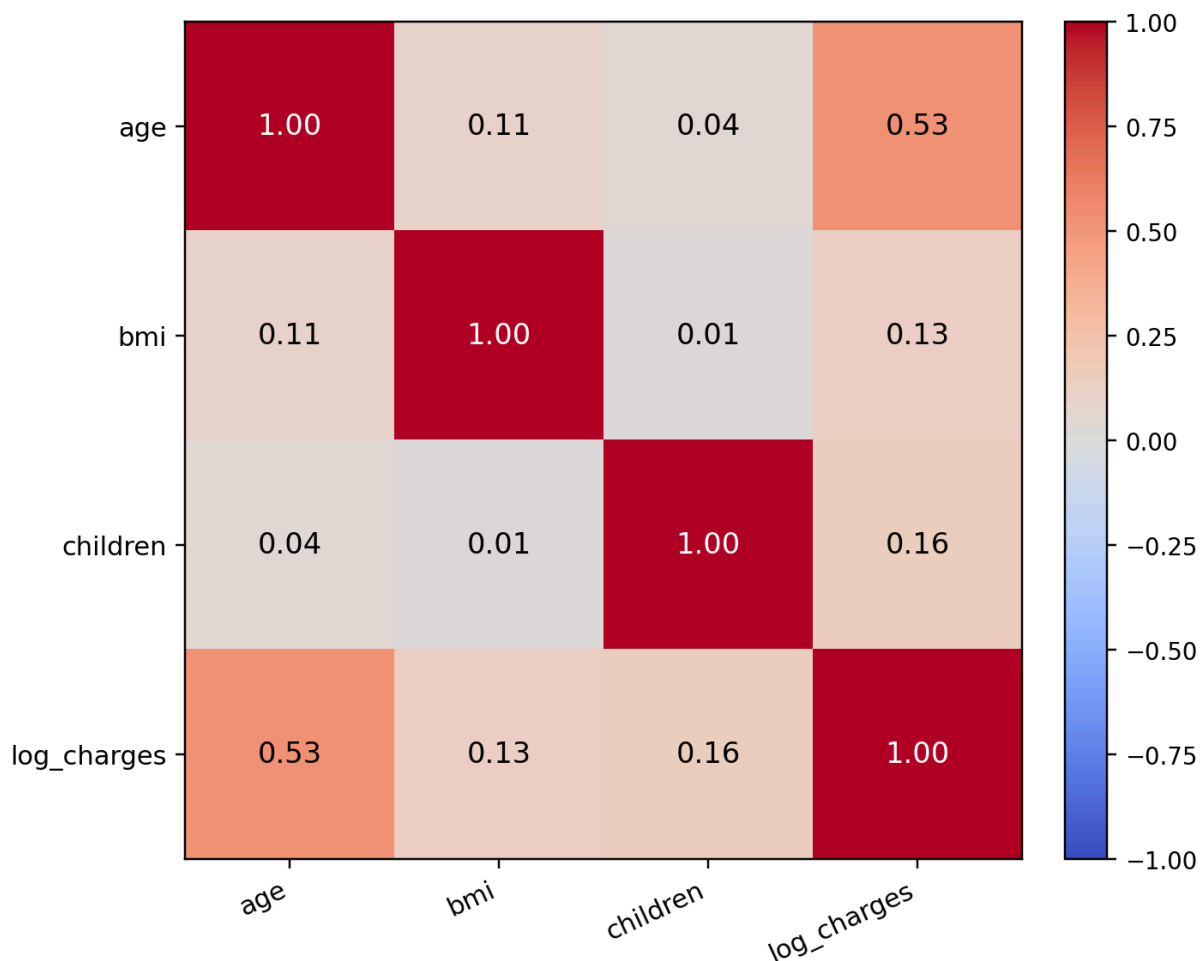


Рисунок 5, б - Корреляционная матрица с $\log(\text{charges} + 1)$

После логарифмирования связь возраста (age) с целевой переменной возрастает примерно до 0,53. Линейная связь с индексом массы тела (bmi) и количеством детей (children) остается слабой. Это не исключает их влияния, а может говорить о нелинейном характере связи, который далее проверяется моделями на основе деревьев.

2.4. Кодирование и разделение данных

Категориальные признаки (sex), (smoker) и (region) были преобразованы с помощью OneHotEncoder. Кодировщик обучался только на тренировочной части, что исключает перенос информации из validation и test в этап подготовки признаков. [2], [4]

Стандартная стратификация для непрерывной целевой переменной невозможна. Поэтому упорядоченные значения (charges) были разбиты на пять интервалов (bins) с примерно одинаковым количеством объектов. Номер интервала объединялся с категорией (smoker), благодаря чему в train, validation и test сохранялись доля курильщиков и представительство разных диапазонов стоимости. Фрагмент кода разделения приведен на рисунке 6, а.

```
df['charges_bin'] = pd.qcut(df['charges'], q=5, duplicates='drop')
df['strata'] = df['charges_bin'].astype(str) + '_' + df['smoker']

train_df, temp_df = train_test_split(
    df, test_size=0.40, random_state=42, stratify=df['strata']
)
validation_df, test_df = train_test_split(
    temp_df, test_size=0.50, random_state=42, stratify=temp_df['strata']
)
```

Рисунок 6, а - Фрагмент кода комбинированного стратифицированного разделения

Сначала 60% наблюдений выделяются в train, затем оставшиеся 40% делятся пополам на validation и test. На обоих этапах пропорции комбинированных групп сохраняются с помощью параметра (stratify).

Таблица 2 - Размеры итоговых выборок

Выборка	Количество объектов	Доля
Train	801	60%
Validation	268	20%
Test	268	20%

age	bmi	children	sex_male	smoker_yes	r1	r2	r3	charges
60.0	29.64	0.0	1.0	0.0	0.0	0.0	0.0	9.452
56.0	33.63	0.0	1.0	1.0	1.0	0.0	0.0	10.69
23.0	27.36	1.0	1.0	0.0	1.0	0.0	0.0	7.934
18.0	34.1	0.0	1.0	0.0	0.0	1.0	0.0	7.037
28.0	26.98	2.0	1.0	0.0	0.0	0.0	0.0	8.398
53.0	26.6	0.0	0.0	0.0	1.0	0.0	0.0	9.245

Рисунок 6, б - Фрагмент датасета после предобработки

После OneHotEncoding сформировано 8 входных признаков. Обозначения регионов: (r1) - northwest, (r2) - southeast, (r3) - southwest; регион northeast является базовой категорией и задается нулями во всех трех столбцах. Поле (charges) содержит логарифмированный таргет, а исходная стоимость хранится в (charges_original).

Таким образом, после предобработки получен набор из 1337 уникальных наблюдений без пропусков, приведенный к единому числовому представлению. Комбинированная стратификация обеспечила близкую структуру train, validation и test, а содержательно объяснимые дорогие случаи были сохранены.

3. Базовая модель и проблема переобучения

3.1. Обучение дерева решений

Дерево решений формирует прогноз с помощью последовательных разбиений пространства признаков. В каждом узле алгоритм выбирает локально лучшее условие по критерию ошибки. Такая модель хорошо интерпретируется, но без ограничений может создавать листья для отдельных наблюдений и запоминать тренировочную выборку. [1], [2]

Неограниченное дерево достигло глубины 20. На train оно получило $R^2 = 0.9992$ и $RMSE = 348.18$, тогда как на test R^2 снизился до 0.6617, а $RMSE$ вырос до 6 635.40. Разрыв R^2 составил 0.3375, что указывает на выраженное переобучение.

$$\text{Ошибка} = \text{Bias}^2 + \text{Variance} + \text{Noise} \quad (7)$$

Неограниченное дерево имеет низкое смещение, но высокий разброс. Ограничение глубины и размеров листьев немного увеличивает смещение, зато снижает чувствительность к конкретному составу train.

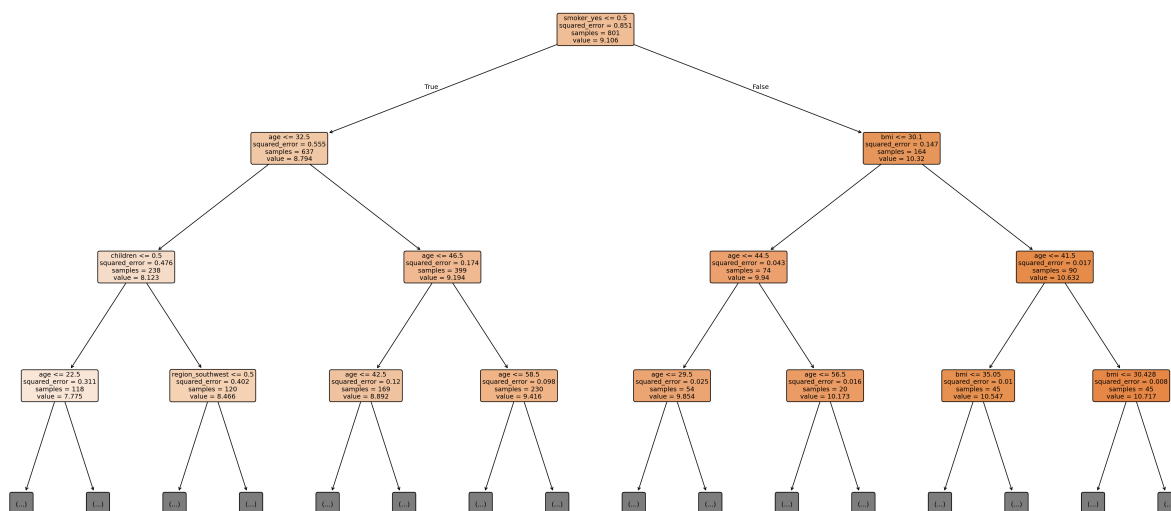


Рисунок 7 - Первые три уровня оптимизированного дерева решений

В каждом узле показано условие разбиения, а движение по ветвям последовательно сужает группу объектов. Для читаемости приведены только первые три уровня полного дерева.

3.2. Оптимизация гиперпараметров

Для настройки применялись GridSearchCV и пятиблочная кросс-валидация KFold с перемешиванием. Сначала широкая сетка определяла перспективную область, затем значения глубины и размеров узлов проверялись с шагом 1 рядом с найденным оптимумом. Validation в поиске не участвовала и использовалась для независимой проверки конфигурации.

```

coarse_grid = {
    'max_depth': [2, 4, 6, 8, 10, 12, 16, None],
    'min_samples_split': [2, 10, 20, 40],
    'min_samples_leaf': [1, 4, 8, 16],
    'criterion': ['squared_error', 'absolute_error'],
    'ccp_alpha': [0.0, 0.0001]
}
cv = KFold(n_splits=5, shuffle=True, random_state=42)
grid = GridSearchCV(tree, coarse_grid, cv=cv, scoring=r2_original_scorer)
  
```

Лучшие параметры DecisionTreeRegressor

Параметр	Значение
criterion	absolute_error
max_depth	10
min_samples_split	2
min_samples_leaf	8

Рисунок 9 - Лучшие гиперпараметры оптимизированного дерева решений

После двухэтапного поиска выбрана глубина 10. Параметры минимального размера узлов и стоимость отсечения (ccp_alpha) ограничивают дробление дерева и уменьшают его разброс.

Переобучение дерева до и после оптимизации

Модель	Выборка	R2	MAE	RMSE
Дерево без ограничений	Train	0.9992	15.61	348.18
Дерево без ограничений	Validation	0.6741	3 474.72	7 043.96
Дерево без ограничений	Test	0.6617	3 185.86	6 635.40
Оптимизированное дерево	Train	0.8576	1 705.66	4 622.26
Оптимизированное дерево	Validation	0.8785	1 564.36	4 301.43
Оптимизированное дерево	Test	0.8399	1 723.63	4 564.54

Рисунок 8 - Метрики дерева до и после оптимизации

После оптимизации R2 составил 0.8576 на train, 0.8785 на validation и 0.8399 на test. Ограничения почти не ухудшили test, но резко сократили разрыв с train, то есть убрали запоминание отдельных наблюдений.

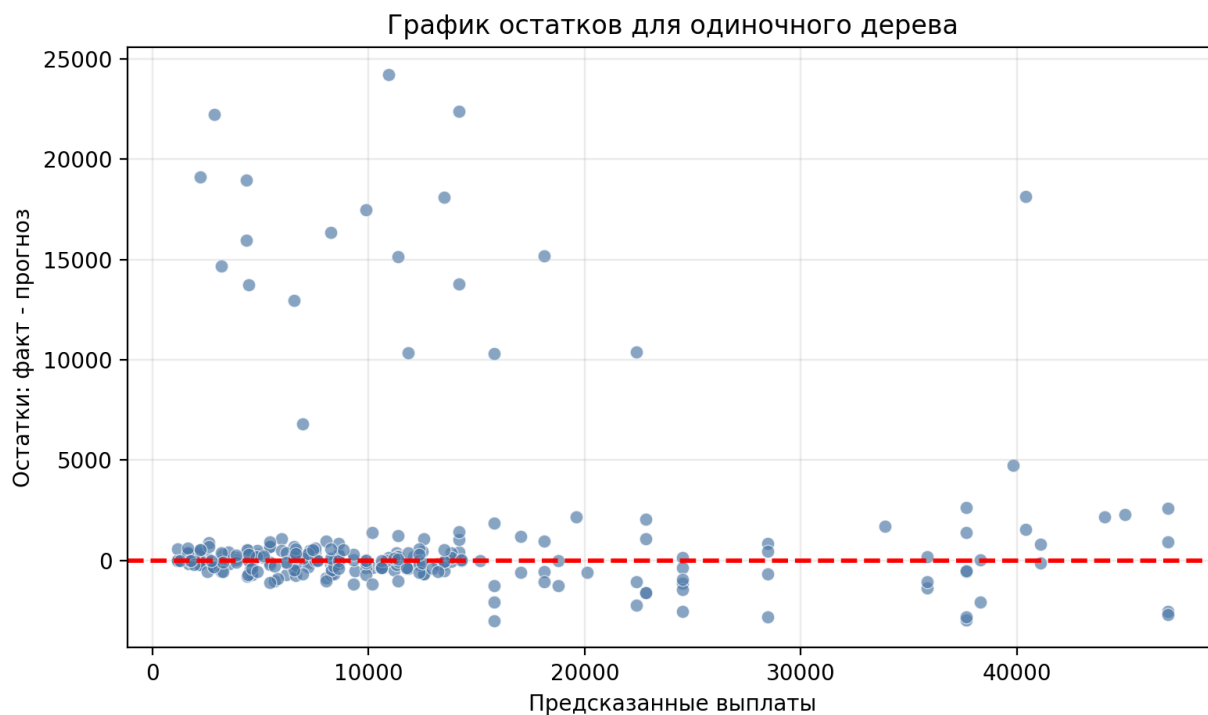


Рисунок 10 - Остатки оптимизированного дерева на тестовой выборке

Большинство остатков сосредоточено около нуля, поэтому модель улавливает основную зависимость. Разброс увеличивается для дорогих полисов, а полосы точек возникают из-за одинакового прогноза для всех объектов одного листа. Качество приемлемое, но редкие крупные ошибки сохраняются.

Двухэтапная настройка сохранила интерпретируемость дерева и заметно уменьшила переобучение. Однако чувствительность к дорогим случаям показала, что дальнейшее улучшение целесообразно искать в ансамблях, снижающих разброс одиночной модели.

4. Ансамблевые модели: Bagging и Random Forest

В Bagging каждое дерево обучается на собственной bootstrap-подвыборке, после чего прогнозы усредняются. Random Forest дополнительно ограничивает набор признаков, доступных при поиске разбиения. Это делает отдельные деревья менее похожими друг на друга и повышает устойчивость совместного прогноза. Для корректного сравнения использовались те же признаки, логарифмированный таргет и разбиение данных. [1], [2]

4.1. Оптимизация Bagging

Двухэтапный GridSearchCV подбирал число деревьев (n_estimators), долю объектов (max_samples), долю признаков (max_features), глубину и минимальные размеры узлов базового дерева. После широкого поиска уточненная сетка строилась около лучших значений.

```
bagging_grid = {
    'n_estimators': [50, 100, 200],
    'max_samples': [0.6, 0.8, 1.0],
    'max_features': [0.6, 0.8, 1.0],
    'estimator__max_depth': [4, 6, 8, 10, None],
    'estimator__min_samples_split': [2, 10, 20],
    'estimator__min_samples_leaf': [1, 4, 8]
}
```

Лучшие параметры BaggingRegressor

Параметр	Значение
n_estimators	50
max_samples	0.6
max_features	1.0
bootstrap	True
estimator__max_depth	5
estimator__min_samples_split	2
estimator__min_samples_leaf	2

Рисунок 11 - Лучшие гиперпараметры Bagging

Итоговая конфигурация определяет разнообразие базовых деревьев и объем данных для каждого из них. На test модель получила $R^2 = 0.8475$ и $RMSE = 4\,454.60$, улучшив оба показателя относительно оптимизированного дерева.

4.2. Оптимизация Random Forest

Random Forest оптимизировался по той же двухэтапной схеме. Помимо числа и глубины деревьев, параметр (max_features) задавал случайную долю признаков, доступных при разбиении, а (bootstrap) включал выбор объектов с возвращением.

```
rf_grid = {
    'n_estimators': [100, 200],
    'max_depth': [4, 6, 8, 10, 12, None],
    'min_samples_split': [2, 10, 20],
    'min_samples_leaf': [1, 4, 8],
    'max_features': ['sqrt', 0.7, 1.0],
    'bootstrap': [True],
    'ccp_alpha': [0.0, 0.0001]
}
```

Лучшие параметры RandomForestRegressor

Параметр	Значение
n_estimators	100
max_depth	6
min_samples_split	2
min_samples_leaf	2
max_features	0.8
bootstrap	True
ccp_alpha	0.0

Рисунок 12 - Лучшие гиперпараметры Random Forest

Параметр (`max_features`) управляет декорреляцией деревьев: слишком малое значение повышает разнообразие, но может ухудшить отдельные разбиения. На test Random Forest получил $R^2 = 0.8485$ и $RMSE = 4\,440.05$.

4.3. Сравнение OOB-score и KFold

OOB-score (out-of-bag score) - это внутренняя оценка качества bootstrap-ансамбля. Каждое дерево обучается на случайной выборке объектов с возвращением, поэтому часть наблюдений в его обучающую подвыборку не попадает. Для каждого объекта прогноз усредняется только по тем деревьям, которые этот объект не использовали при обучении; полученная оценка по смыслу напоминает проверку на отложенных данных и не требует выделять еще одну выборку. Чем ближе OOB R^2 к результатам KFold и test, тем устойчивее выглядит оценка качества модели. Для одиночного дерева OOB-score не определен, поскольку оно не является bootstrap-композицией. [2]

```
bagging_oob = BaggingRegressor(  
    estimator=best_tree, n_estimators=best_n,  
    bootstrap=True, oob_score=True, random_state=42  
)  
  
rf_oob = RandomForestRegressor(  
    n_estimators=best_n, bootstrap=True,  
    oob_score=True, random_state=42  
)
```

Сравнение OOB, KFold и test

Модель	OOB R2	KFold R2	Test R2
Оптимизированное дерево	-	0.8319	0.8399
Bagging	0.8628	0.8478	0.8475
Random Forest	0.8637	0.8466	0.8485

Рисунок 13 - Сравнение оценок OOB, KFold и test

Для Bagging OOB R2 равен 0,8628 против 0,8478 по KFold, для Random Forest - 0,8637 против 0,8466. Тестовые значения 0,8475 и 0,8485 близки к KFold, поэтому общий вывод о качестве не зависит от одной процедуры валидации.

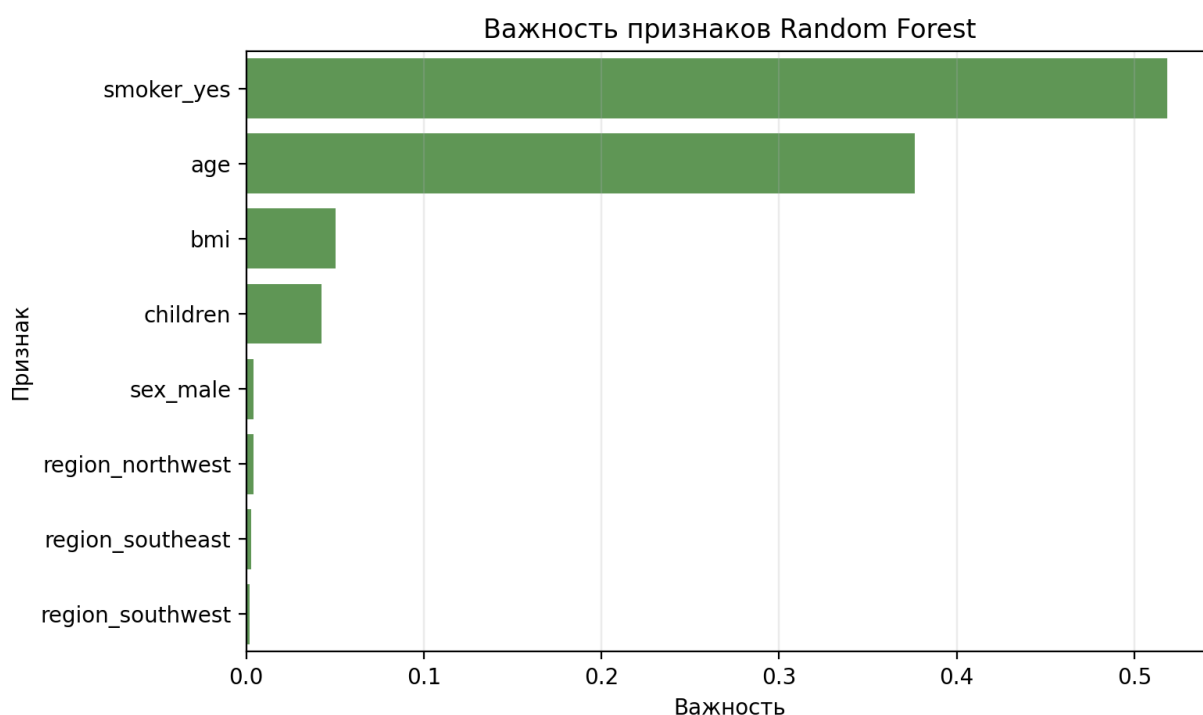


Рисунок 14 - Важность признаков Random Forest

Наибольший вклад в Random Forest имеют факт курения (smoker_yes) и возраст (age). Индекс массы тела (bmi) занимает третье место, а пол и регион влияют слабо. Результат согласуется с предварительным анализом данных.

Bagging и Random Forest улучшили R^2 и RMSE относительно оптимизированного дерева, сохранив близкие результаты на train, validation и test. Небольшая разница между ансамблями объясняется сильным влиянием курения: даже при случайном выборе признаков модели часто приходят к похожим ключевым разбиениям.

5. Градиентный бустинг: XGBoost

В отличие от Bagging и Random Forest, градиентный бустинг строит деревья последовательно: каждое новое дерево направлено на исправление ошибок уже сформированного ансамбля. В работе использовалась модель XGBRegressor из библиотеки XGBoost. [1], [3]

На каждом шаге вычисляются псевдоостатки, задающие направление уменьшения функции потерь. Для квадратичной ошибки они совпадают с обычными остатками. Новое дерево приближает эти значения, а величина его вклада регулируется скоростью обучения (learning_rate).

$$r_i^{(m)} = -\partial L(y_i, F(x_i)) / \partial F(x_i) \quad (8)$$

$$F_m(x) = F_{m-1}(x) + \eta h_m(x) \quad (9)$$

5.1. Двухэтапная оптимизация гиперпараметров

Сначала RandomizedSearchCV проверил 60 комбинаций на пяти блоках KFold, затем еще 50 комбинаций были исследованы около лучшего решения. Средний CV R2 вырос с 0.8399 до 0.8426; наилучшее качество на validation достигнуто примерно после 648 деревьев.

```
xgb_grid = {
    'n_estimators': [150, 250, 400, 600, 800],
    'learning_rate': [0.01, 0.02, 0.03, 0.05, 0.08],
    'max_depth': [2, 3, 4, 5],
    'min_child_weight': [1, 3, 5, 8],
    'subsample': [0.7, 0.8, 0.9, 1.0],
    'colsample_bytree': [0.7, 0.8, 0.9, 1.0],
    'reg_lambda': [0.5, 1, 3, 5, 10],
    'reg_alpha': [0, 0.01, 0.1, 0.5]
}
```

Лучшие параметры XGBoost

Гиперпараметр	Значение
subsample	0.9
reg_lambda	4.0
reg_alpha	0.2
n_estimators	700
min_child_weight	5
max_depth	3
learning_rate	0.01
colsample_bytree	1.0

Рисунок 15 - Лучшие гиперпараметры XGBoost

Небольшая скорость обучения ($\text{learning_rate} = 0,01$), глубина 3 и регуляризация обеспечивают постепенное улучшение прогноза без чрезмерно сложных отдельных деревьев.

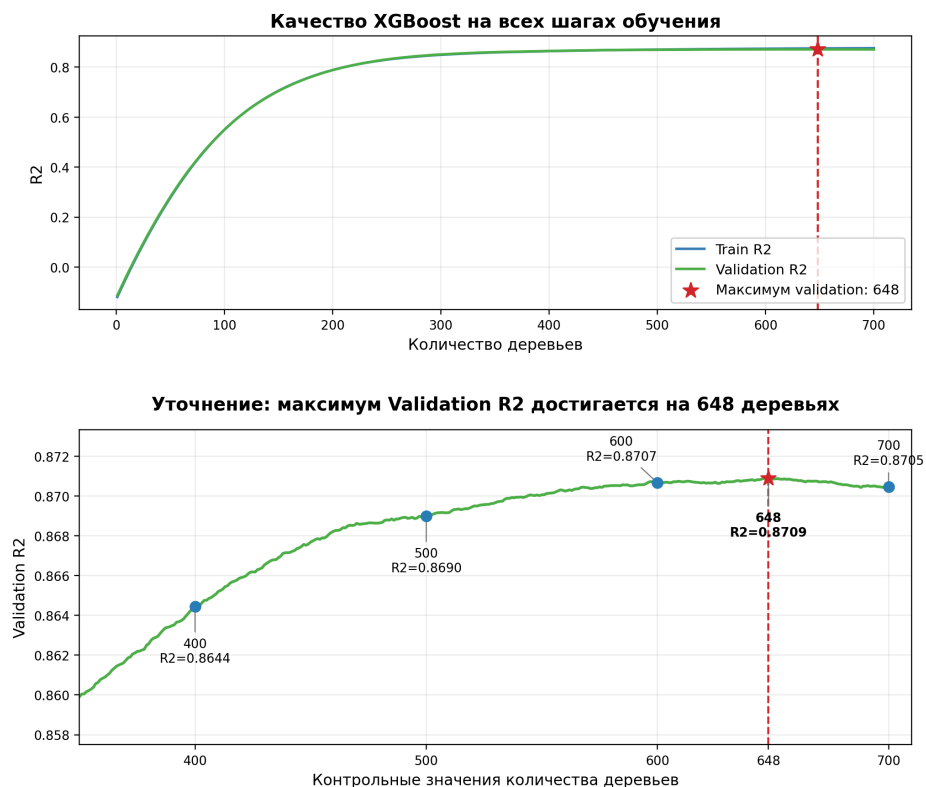


Рисунок 16 - Обоснование выбора 648 деревьев для XGBoost

На увеличенном участке показаны контрольные значения Validation R2: 400 деревьев - 0.8644, 500 - 0.8690, 600 - 0.8707, 648 - 0.8709, 700 - 0.8705. Качество растет до 648 деревьев, где достигается максимум, а затем немного снижается; поэтому выбрано именно 648 деревьев.

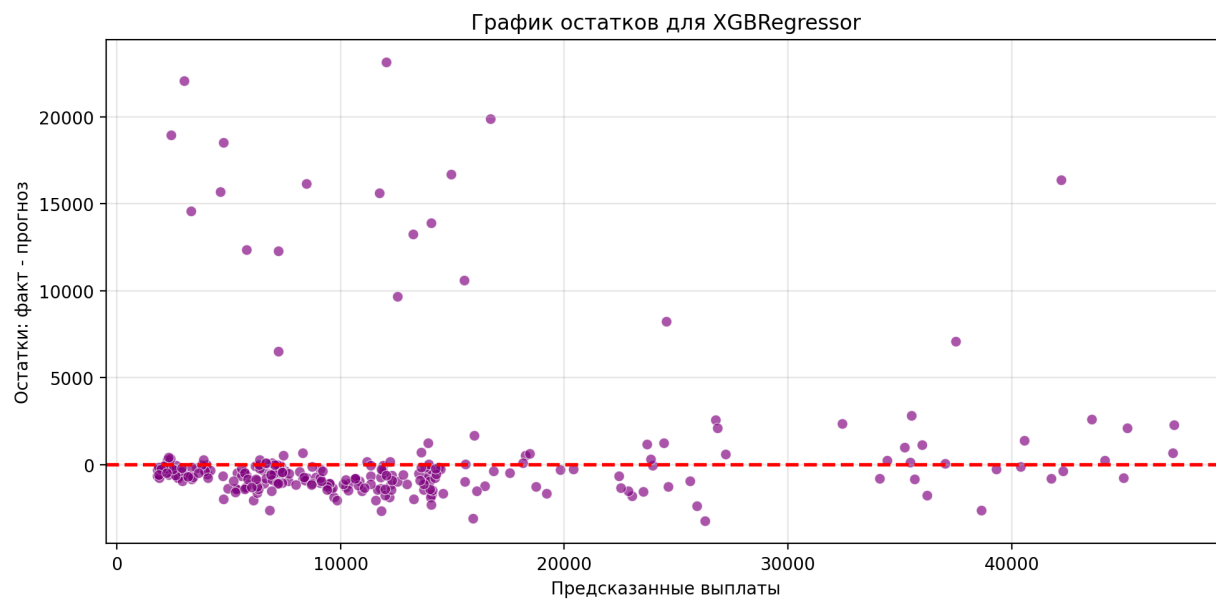


Рисунок 17 - Остатки XGBoost на тестовой выборке

Облако остатков в основном центрировано около нуля, поэтому общего систематического смещения не наблюдается. Вместе с тем разброс возрастает у дорогих полисов: модель адекватна, но неоднородность ошибок полностью не устранена.

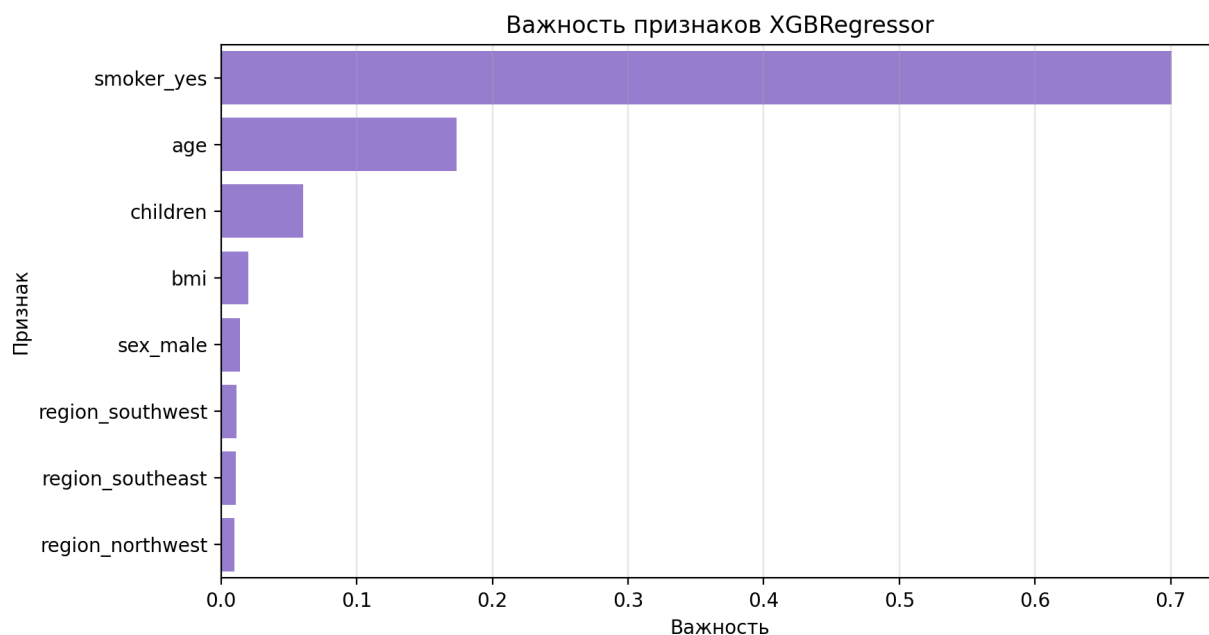


Рисунок 18 - Важность признаков XGBoost

Признак курения (`smoker_yes`) формирует около 70% суммарной важности. Далее следуют возраст (`age`), количество детей (`children`) и индекс массы тела (`bmi`). Это подтверждает гипотезу EDA о сильном влиянии курения.

На test XGBoost достиг $R^2 = 0.8579$ и $RMSE = 4\,300.09$, показав лучший результат по этим двум метрикам. Близкие значения R^2 на train (0.8733), validation (0.8914) и test не указывают на выраженное переобучение.

6. Сравнительный анализ моделей

Итоговое сравнение выполнялось на неизменных выборках. Test не участвовала ни в обучении, ни в подборе гиперпараметров и поэтому служила основной оценкой обобщающей способности. Помимо метрик анализировались остатки, фактические и предсказанные значения, важность признаков, SHAP и скорость инференса.

Таблица 3 - Метрики моделей на train, validation и test

Модель	Выборка	R2	MAE	MSE	RMSE	MAPE
Оптимизированное дерево	Train	0.8576	1 705.66	2.14e+07	4 622.26	9.90%
Оптимизированное дерево	Validation	0.8785	1 564.36	1.85e+07	4 301.43	9.71%
Оптимизированное дерево	Test	0.8399	1 723.63	2.08e+07	4 564.54	10.38%
Bagging	Train	0.8756	1 954.27	1.87e+07	4 319.79	16.32%
Bagging	Validation	0.8918	1 817.67	1.65e+07	4 058.75	15.91%
Bagging	Test	0.8475	2 010.78	1.98e+07	4 454.60	17.11%
Random Forest	Train	0.8855	1 838.33	1.72e+07	4 144.22	15.39%
Random Forest	Validation	0.9006	1 683.34	1.51e+07	3 889.85	14.77%
Random Forest	Test	0.8485	1 964.76	1.97e+07	4 440.05	16.91%
XGBoost	Train	0.8733	1 962.75	1.90e+07	4 359.46	15.82%
XGBoost	Validation	0.8914	1 805.61	1.65e+07	4 065.35	14.63%
XGBoost	Test	0.8579	1 885.83	1.85e+07	4 300.09	15.87%

Таблица 3 позволяет одновременно сравнить итоговое качество и разрыв между выборками. Все значения рассчитаны в исходной шкале после преобразования $\exp(\text{prediction}) - 1$.

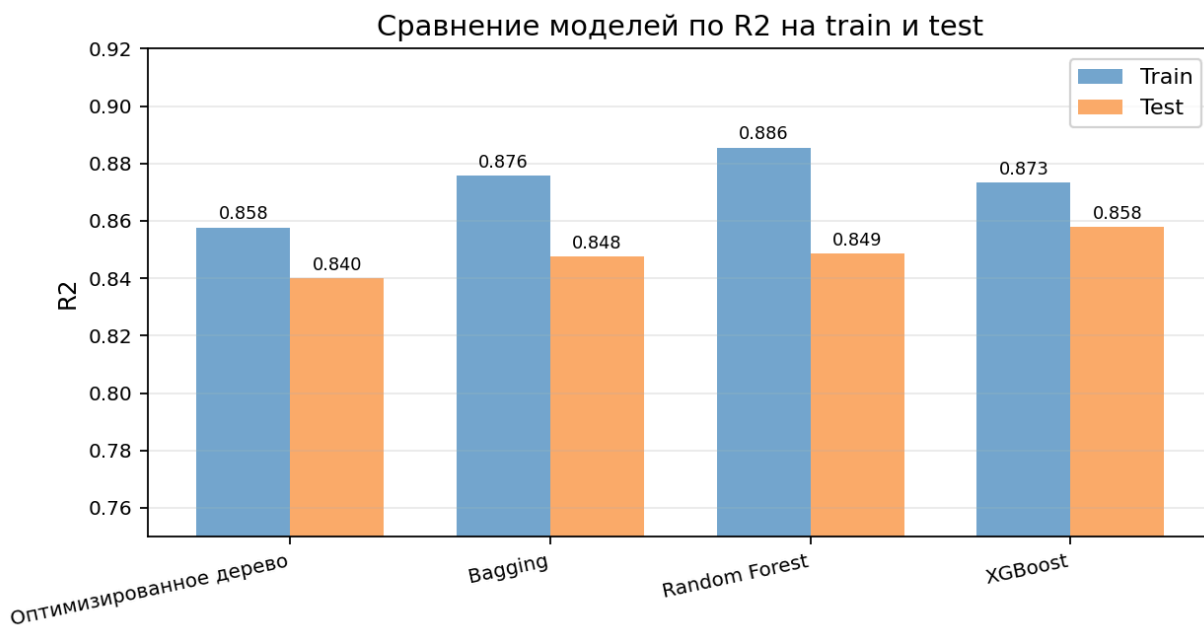


Рисунок 19 - Сравнение моделей по R2 на train и test

У всех оптимизированных моделей разрыв между train и test остается умеренным. Наибольший test R2 показывает XGBoost, тогда как у Random Forest различие между train и test немного больше, чем у остальных ансамблей.

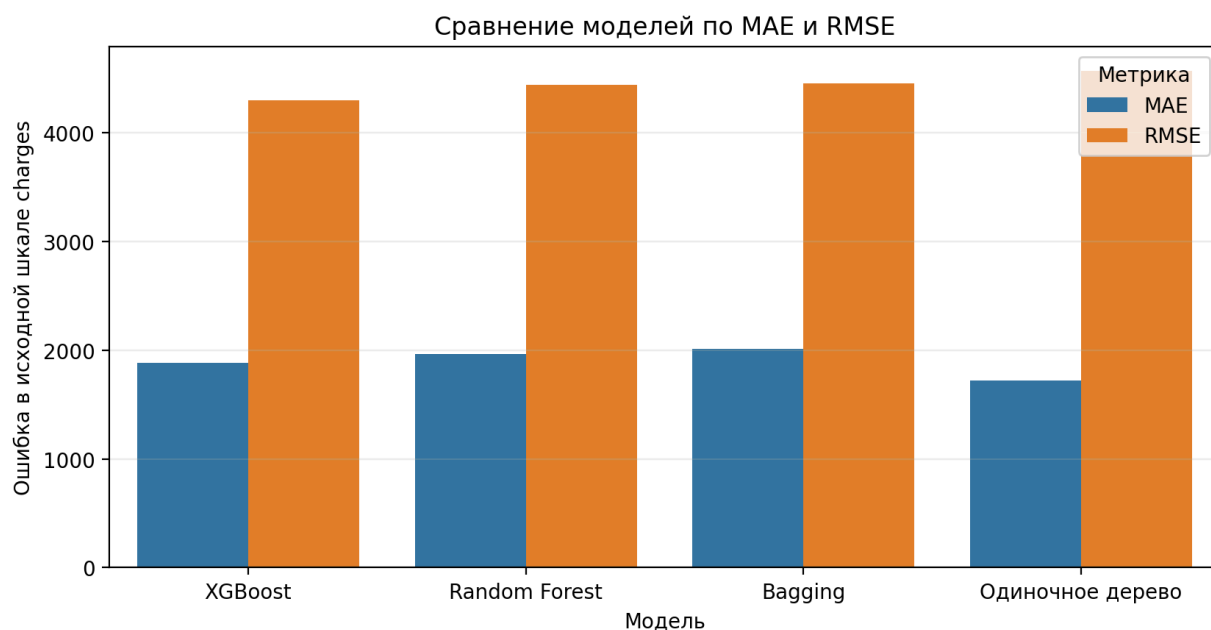


Рисунок 20 - Сравнение MAE и RMSE на тестовой выборке

RMSE находится в узком диапазоне примерно от 4 300 до 4 565 долларов. Все модели используют одни признаки и сталкиваются с одинаковыми редкими дорогими случаями, которые сильнее всего влияют на квадрат ошибки. У оптимизированного дерева MAE ниже, но RMSE выше, то есть большинство прогнозов близки к факту, однако отдельные крупные промахи сильнее ухудшают результат.

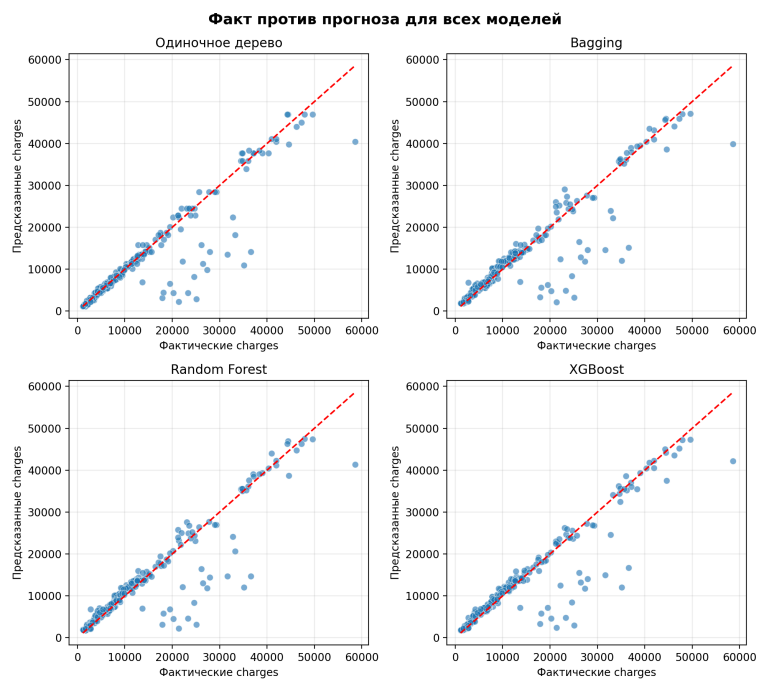


Рисунок 21 - Предсказанные и фактические значения для всех моделей

Чем ближе точки к диагонали, тем точнее прогноз. XGBoost формирует наиболее компактное облако, но в верхнем диапазоне стоимости все модели чаще отклоняются от идеальной линии.

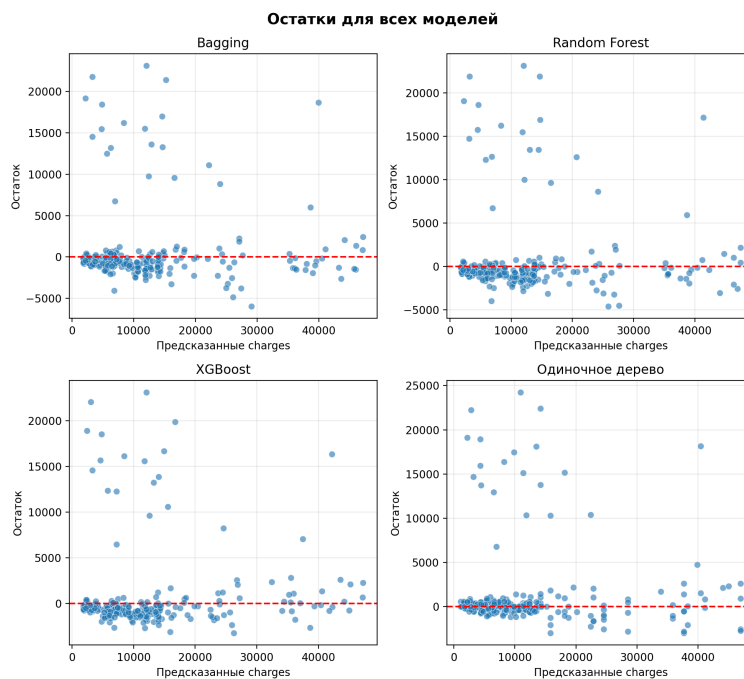


Рисунок 22 - Остатки моделей на тестовой выборке

У ансамблей облака остатков более сглажены, чем у одиночного дерева, а постоянного смещения относительно нуля не видно. Расширение облака справа подтверждает, что дорогие полисы остаются общей сложной областью.

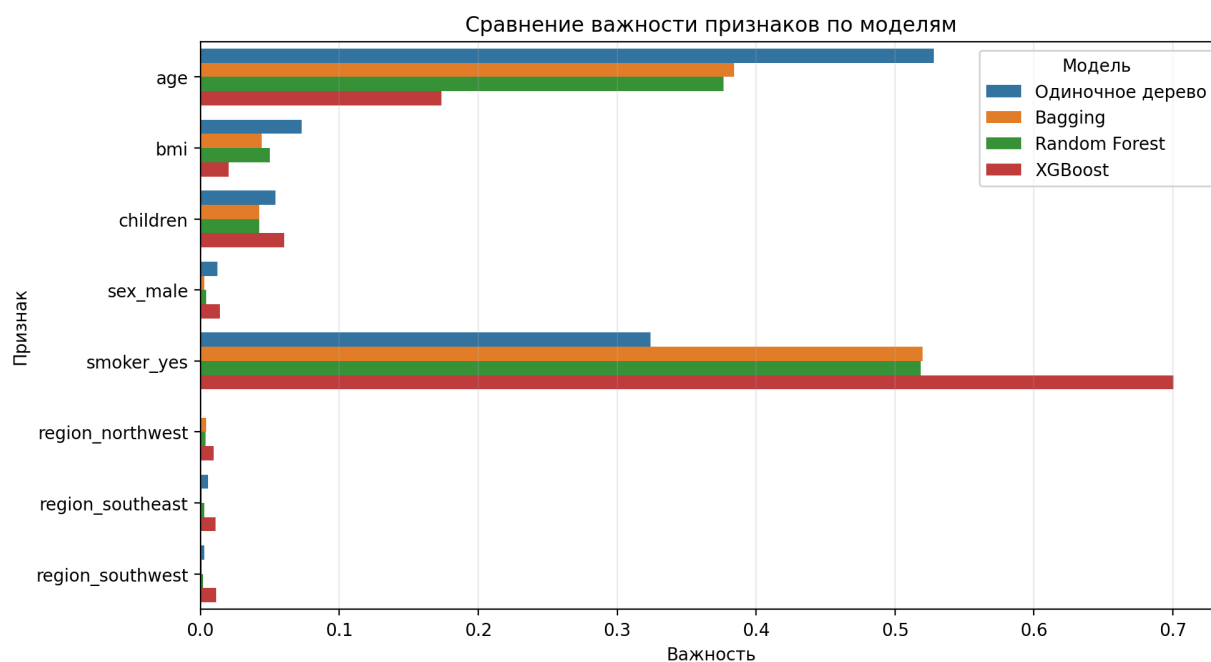


Рисунок 23 - Сравнение важности признаков

Во всех ансамблях главным фактором остается курение (smoker_yes), далее обычно следуют возраст (age) и индекс массы тела (bmi). Пол и регион практически не влияют на целевую переменную, что согласуется с предварительным анализом.

SHAP (SHapley Additive exPlanations) - метод интерпретации, который раскладывает отдельный прогноз модели на базовое значение и вклады признаков. Положительное SHAP-значение повышает прогноз относительно базового уровня, отрицательное - понижает, а абсолютная величина показывает силу влияния признака для конкретного объекта. Обобщение таких значений по всей выборке позволяет оценить глобальную важность и направление воздействия признаков. [8]

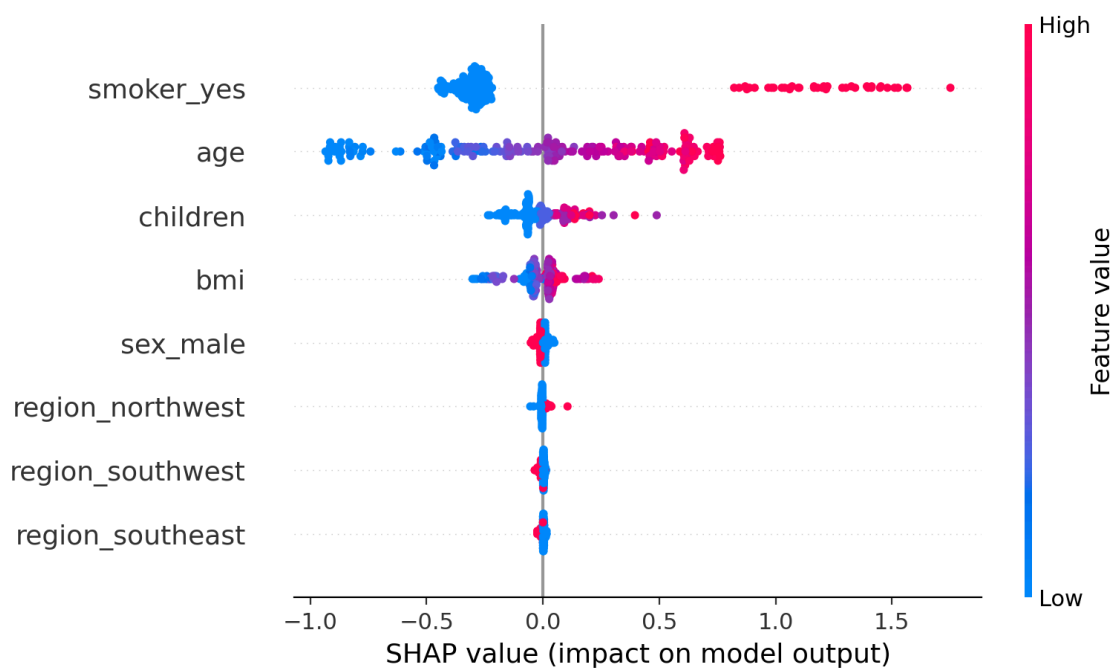


Рисунок 24 - SHAP-анализ модели Random Forest

SHAP показывает не только силу, но и направление влияния. Курение и больший возраст чаще сдвигают прогноз $\log(\text{charges} + 1)$ вверх, что дополняет встроенную важность признаков содержательной интерпретацией.

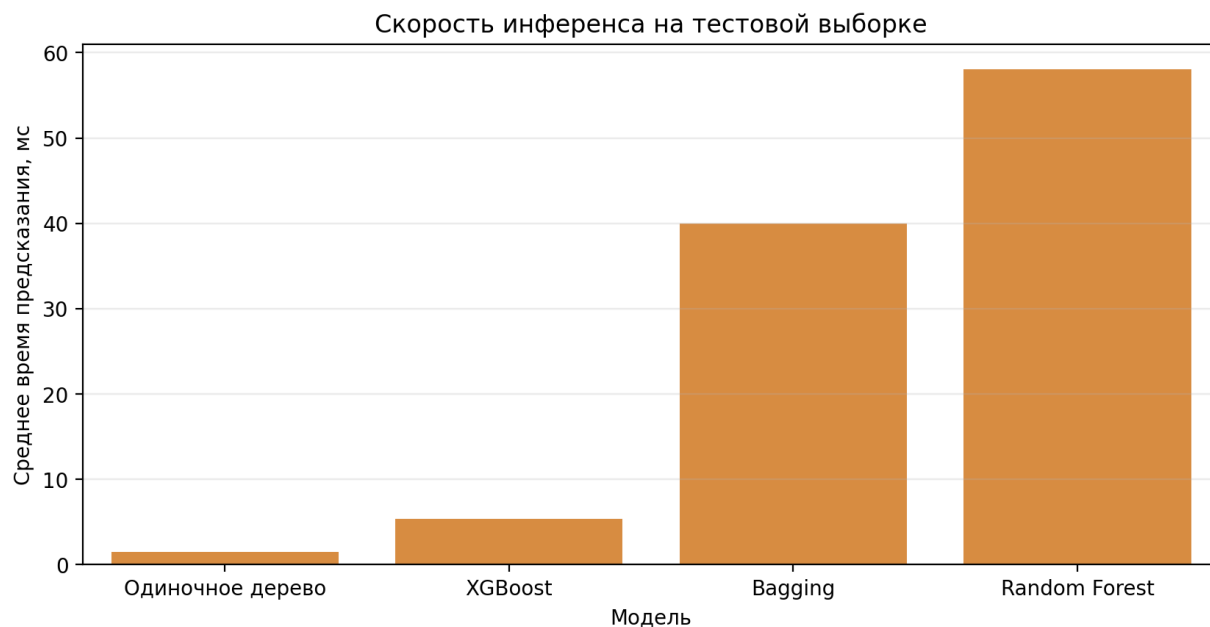


Рисунок 25 - Среднее время инференса моделей

Одиночное дерево прогнозирует быстрее всего, XGBoost занимает промежуточное положение, а Bagging и Random Forest требуют больше времени из-за вычисления прогнозов множества деревьев. Для данного небольшого датасета все измеренные значения остаются практически приемлемыми.

Заключение

В ходе работы был исследован набор из 1338 наблюдений для прогнозирования стоимости медицинской страховки (charges). Пропущенных значений не обнаружено, одна полностью повторяющаяся строка удалена. Категориальные признаки преобразованы методом OneHotEncoding, а данные разделены на train, validation и test с комбинированной стратификацией по интервалам стоимости и признаку курения (smoker).

EDA выявил правостороннюю асимметрию целевой переменной: ее коэффициент составил 1.52. Среди 133 полисов дороже 35 000 долларов доля курильщиков достигла 97.7%, поэтому высокие значения были признаны содержательно объяснимыми и сохранены. Логарифмирование уменьшило асимметрию до -0.09.

На test оптимизированное дерево получило $R^2 = 0.8399$, Bagging - 0.8475, Random Forest - 0.8485, а XGBoost - 0.8579. Лучшей по R^2 и RMSE стала модель XGBoost: RMSE составил 4 300.09, MAE - 1 885.83. При этом оптимизированное дерево показало меньшие MAE (1 723.63) и MAPE (10.38%), поэтому выбор модели зависит от того, насколько критичны отдельные крупные ошибки.

Для XGBoost значения R^2 на train, validation и test равны 0.8733, 0.8914 и 0.8579. Небольшой разрыв не указывает на выраженное переобучение, а test R^2 означает, что модель объясняет около 85.8% вариации стоимости на ранее не использованных данных. Основная часть оставшейся ошибки связана с наиболее дорогими полисами.

Интерпретация лучшей модели согласуется с EDA. Признак курения (smoker_yes) формирует около 70% встроенной важности XGBoost, подтверждая концентрацию дорогих полисов среди курильщиков. Возраст (age) также остается значимым фактором и соответствует наиболее сильной корреляции с $\log(\text{charges} + 1)$. Индекс массы тела (bmi) и количество детей (children) дают меньший, преимущественно нелинейный вклад, а пол (sex) и регион (region) почти не влияют на прогноз.

В последующих работах настройку XGBoost можно сделать более глубокой. Для этого целесообразно увеличить число проверяемых комбинаций и расширить диапазоны количества деревьев (`n_estimators`), скорости обучения (`learning_rate`), глубины (`max_depth`), минимального веса дочернего узла (`min_child_weight`), долей объектов и признаков (`subsample`, `colsample_bytree`), а также коэффициентов регуляризации (`reg_alpha`, `reg_lambda`). После широкого поиска перспективную область следует исследовать с меньшим шагом либо применить байесовскую оптимизацию. Раннее прекращение обучения (`early stopping`) позволит выбирать число деревьев по качеству на `validation`, а повторная или вложенная кросс-валидация снизит зависимость результата от одного разбиения данных.

Дополнительный резерв качества связан с данными и постановкой задачи. Полезно добавить сведения о хронических заболеваниях, типе страхового плана, уровне дохода и истории обращений, проверить взаимодействия возраста (`age`), курения (`smoker`) и индекса массы тела (`bmi`), а также отдельно настроить функцию потерь для дорогих полисов. Можно сравнить обучение на исходной и логарифмической шкале, робастные функции потерь и интервальные прогнозы. Каждое улучшение необходимо подтверждать на одной и той же отложенной тестовой выборке, а итоговую модель дополнительно проверять на внешних или более поздних данных.

Итоговой моделью по совокупности R^2 и RMSE выбран XGBoost. Оптимизированное дерево остается наиболее простым и быстрым решением, Random Forest дает близкое качество и удобен для OOB- и SHAP-анализа, однако XGBoost лучше всего сокращает крупные ошибки и обеспечивает наиболее высокий test R^2 .

Список использованных источников

1. Яндекс Образование. Учебник по машинному обучению: решающие деревья, ансамбли, градиентный бустинг, метрики и кросс-валидация. URL: <https://education.yandex.ru/handbook/ml>.
2. Scikit-learn documentation. DecisionTreeRegressor, BaggingRegressor, RandomForestRegressor, OneHotEncoder, GridSearchCV и RandomizedSearchCV. URL: <https://scikit-learn.org/stable/>.
3. XGBoost documentation. XGBRegressor and gradient boosted decision trees. URL: <https://xgboost.readthedocs.io/en/stable/>.
4. Pandas documentation. Работа с табличными данными и расчет описательных статистик. URL: <https://pandas.pydata.org/docs/>.
5. NumPy documentation. Численные преобразования, включая log1p и expm1. URL: <https://numpy.org/doc/stable/>.
6. Matplotlib documentation. Построение и оформление графиков. URL: <https://matplotlib.org/stable/>.
7. Seaborn documentation. Статистическая визуализация данных. URL: <https://seaborn.pydata.org/>.
8. SHAP documentation. Интерпретация моделей машинного обучения на основе SHAP-значений. URL: <https://shap.readthedocs.io/en/latest/>.
9. Medical Cost Personal Datasets. Набор данных о стоимости медицинской страховки. URL: <https://www.kaggle.com/datasets/mirichoi0218/insurance>.